

Reviewer Pack — Minimal Local Index in Algebraic Topology and Circuit Depth

Entry d5923b669272 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 10:13:49 UTC

Minimal Local Index in Algebraic Topology and Circuit Depth

Entry ID: d5923b669272 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 10:13:49 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Local Systems) **Field B** (complexity object): Boolean Circuits (Circuit Depth)

Statement:

For every boolean circuit C with depth d , the minimal local index of its associated cell complex is $O(d)$. Equivalently, for any Tseitin formula φ_G corresponding to C , the minimal local index of its vertex set is $\Theta(d \log n)$, where n is the number of variables in φ_G .

Rationale (proposer's reasoning):

Local systems in algebraic topology provide a rich framework for studying the topology of cell complexes. Minimal local index measures the complexity of these systems and can be computed using standard tools from sheaf theory. The conjecture links this invariant to circuit depth, which is a well-studied measure of computational complexity. If true, it would suggest a deep connection between algebraic topology and computational complexity.

Taxonomy category: algebraic-topology-circuit-depth (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7fb23e4a07e205e1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the minimal local index and the circuit depth is ≥ 0.7 for all circuits, and falsified if any single seed produces a Pearson correlation coefficient < 0.5 or any individual vertex's local index exceeds $4d \log n$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic topology AND local systems AND circuit depth - Tseitin formula AND minimal local index IN BOOLEAN CIRCUITS - vertex set minimal local index $\Theta(d \log n)$ algebraic topology

Top relevant hits considered: - [http://arxiv.org/abs/1111.0731v2] On Hawaiian Groups of Some Topological Spaces - [http://arxiv.org/abs/1907.00871v6] Cardinal-indexed classifying spaces for families of subgroups of any topological group - [http://arxiv.org/abs/1511.03526v3] Local duality in algebra and topology - [http://arxiv.org/abs/1109.2049v1] Structure-Based Local Search Heuristics for Circuit-Level Boolean Satisfiability - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/2303.06643v1] General Boolean Formula Minimization with QBF Solvers - [http://arxiv.org/abs/2604.05701v1] Measurement of the CKM angle γ in $B^\pm \rightarrow D(\rightarrow K_S^0 h'^+ h'^-) h^\pm$ dec - [http://arxiv.org/abs/2305.01463v2] Test of lepton flavour universality using $B^0 \rightarrow D^{*-} \tau^+ \nu_\tau$ decays with hadronic τ channels

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_circuit(depth):
        if depth == 1:
            return ['0', '1']
        left = generate_boolean_circuit(depth - 1)
        right = generate_boolean_circuit(depth - 1)
        return [f'({l} AND {r})' for l in left] + [f'({l} OR {r})' for l in right]

    def tseitin_formula(circuit):
        n = len(circuit)
        variables = list(range(n))
        formulas = []
        for i, expr in enumerate(circuit):
            if 'AND' in expr:
                left, right = expr.split(' AND ')
                new_var = f'v{i}'
                formulas.append(f'{new_var} <=> ({left} AND {right})')
                variables.append(new_var)
            elif 'OR' in expr:
```

```

        left, right = expr.split(' OR ')
        new_var = f'v{i}'
        formulas.append(f'{new_var} <=> ({left} OR {right})')
        variables.append(new_var)
    return formulas, variables

def compute_local_index(formulas, variables):
    n = len(variables)
    local_indices = [0] * n
    for formula in formulas:
        if '=>' in formula:
            antecedent, consequent = formula.split(' => ')
            if 'AND' in consequent:
                left, right = consequent.split(' AND ')
                local_indices[variables.index(antecedent)] += 1
                local_indices[variables.index(left)] += 1
                local_indices[variables.index(right)] += 1
    return max(local_indices)

depths = [5, 10, 15, 20, 30, 40]
metric_values = []
instances_tested = 0
n_max = 0

for depth in depths:
    circuit = generate_boolean_circuit(depth)
    formulas, variables = tseitin_formula(circuit)
    local_index = compute_local_index(formulas, variables)

    if local_index > 4 * depth * math.log(len(variables)):
        return {
            "metric_name": "minimal_local_index",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": f"Local index {local_index} exceeds 4d log n for d={depth}, n={len(variables)}"
        }

    metric_values.append(local_index)
    instances_tested += len(circuit)
    n_max = max(n_max, depth)

mean_msl = sum(metric_values) / len(metric_values)
std_dev = math.sqrt(sum((x - mean_msl) ** 2 for x in metric_values) / len(metric_values))

return {
    "metric_name": "minimal_local_index",
    "metric_value": mean_msl,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": all(0.5 <= msl / d for d, msl in zip(depths, metric_values)) and all(n_max <= d for d in depths),
    "counterexample": ""
}

if __name__ == "__main__":

```

```

import sys
seeds = list(map(int, sys.argv[1:])) or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

results = []
for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")
    results.append(trial_result)

mean_msl = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
std_dev = math.sqrt(sum((r["metric_value"] - mean_msl) ** 2 for r in results if r["metric_value"] is not None) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_msl} std={std_dev} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results) and any(r["metric_value"] < 0.5 * d for d in seeds):
    print(f"RESULT: FALSIFIED counterexample=\"local_index_too_small\" first_failing_seed={seeds[0]}")
else:
    print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cdceed41.py", line 100, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cdceed41.py", line 64, in run_trial
    circuit = generate_boolean_circuit(depth)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cdceed41.py", line 24, in generate_boolean_circuit
    left = generate_boolean_circuit(depth - 1)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cdceed41.py", line 24, in generate_boolean_circuit
    left = generate_boolean_circuit(depth - 1)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cdceed41.py", line 24, in generate_boolean_circuit
    left = generate_boolean_circuit(depth - 1)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cdceed41.py", line 26, in generate_boolean_circuit
    return [f'({l} AND {r})' for l in left] + [f'({l} OR {r})' for l in right]
           ^
NameError: name 'r' is not defined. Did you mean: 're'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was unable to complete its execution and provide a result for the conjecture. | next: Investigate the cause of the crash in the test code and fix it to allow for proper testing of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12517
2	preregistration	ollama_remote	glm4:latest	0	0	15244
3	novelty	ollama_remote	glm4:latest	0	0	10976
4	novelty	ollama_remote	glm4:latest	0	0	10061
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	43199
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15929
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18406
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14393
9	judge	ollama_remote	glm4:latest	0	0	17057

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 157783 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/d5923b669272.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/d5923b669272.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/d5923b669272.tar.gz (if generated)